CSC 251 Lecture 12 – Exploratory Data Analysis (EDA), Data Visualization

DataFrame Input and Output

Pandas provides functionality for inputting/outputting tabular data to/from DataFrames using numerous formats:

- dict of arrays, lists, or tuples
- dict of Series
- dict of dicts
- list of dicts or Series
- list of lists or tuples
- another DataFrame
- NumPy masked or structured arrays
- pickled objects
- delimited files (CSV)
- Excel files
- JSON string or structed data
- HTML table
- XML doc
- LaTex table
- SAS, SPSS, SQL, STATA files

The most common, for CSV (comma separated values) files has over 50 options:

```
df = pd.read_csv('datafile.csv')
```

For an extensive list of input and output options that do the conversion to and from DataFrames:
https://pandas.pydata.org/docs/reference/io.html

Exploratory Data Analysis (EDA) Example

Given a set of data, the first step in data analysis to observe general patterns, trends, features and outliers.

- Generate questions about the data
- Search for answers by visualizing, transforming, and modeling the data
- Use what you learn to refine your questions and generate more questions

View `tips.csv` file in a Jupyter notebook and generate questions:

1)

2)

3)

Data Visualization

"Data viz" assists you in:

- Understanding your data more easily
- Recognizing relationships between variables
- Spotting unusual datapoints like outliers
- Telling a story

Libraries

pandas – Provides plot options for Series and DataFrame objects. Uses the matplotlib library and is easier to use but has limited functionality.

matplotlib - A popular plotting and data visualization package designed to create publication-quality plots, intended to enable a MATLAB-like interface, can export visualizations to all common vector and graphics formats (PDF, JPG, PNG, GIF, etc.) A low-level API that gives you a lot of control.

seaborn – A spin-off from matplotlib, considered easier to use than matplotlib and pandas.

plotly – Open source, allows for interactive graphs, charts, visualizations

Getting Started with matplotlib

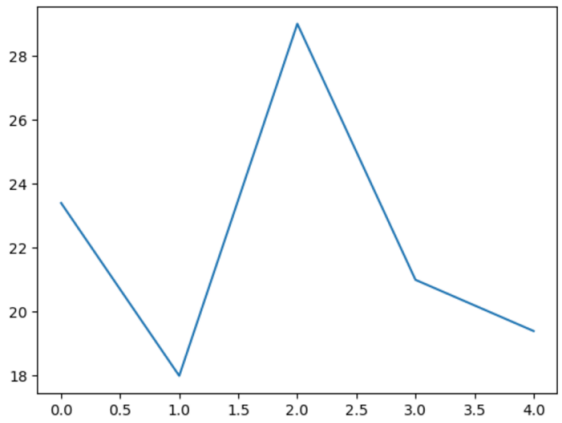
We've already imported and used matplotlib to create simple plots.

```
import matplotlib.pyplot as plt
```

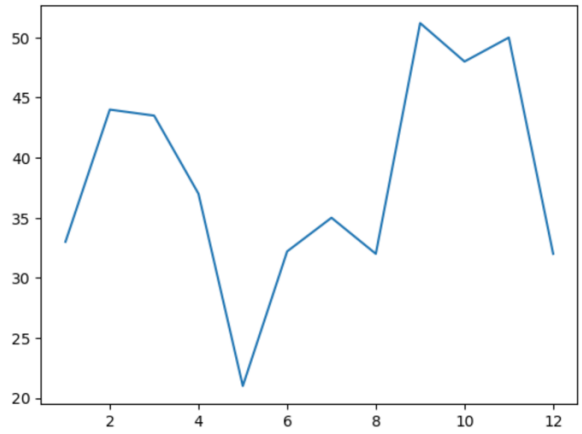
where matplotlib is the package and pyplot is a subpackage or module within matplotlib.

Example 1a,b,c: For a basic matplotlib line plot using a list, ndarray, or Series, the index will be used by default.

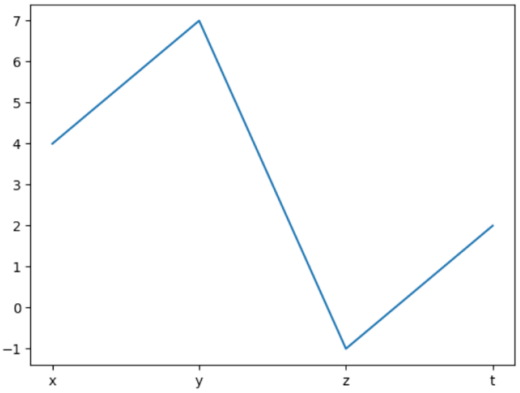
```
data = [23.4,18,29,21,19.4]
plt.plot(data)
```




```
dist = [33,44,43.5,37,21,32.2,35,32,51.2,48,50,32]
num = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
plt.plot(num, dist)
```

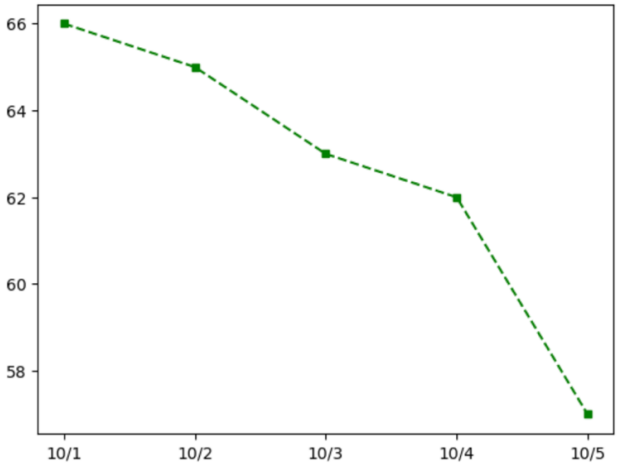



```
data2 = pd.Series([4,7,-1,2], index = ['x','y','z','t'])
plt.plot(data2)
```



Example 1d: Notice the many plotting options:

```
plt.plot(['10/1', '10/2', '10/3', '10/4', '10/5'],[66, 65, 63, 62, 57],color= 'green',
      marker= 's',linestyle= '--', markersize= 5)
```



Matplotlib allows a format string to combine the green dashed-line with square markers into `'g--s'`.

Matplotlib marker options:

marker	symbol	description
"."	•	point
","	·	pixel
"o"	●	circle
"v"	▼	triangle_down
"^"	▲	triangle_up
"<"	◀	triangle_left
">"	▶	triangle_right
"1"	Y	tri_down
"2"	⅄	tri_up
"3"	≺	tri_left
"4"	≻	tri_right
"8"	●	octagon
"s"	■	square
"p"	⬟	pentagon
"P"	✚	plus (filled)
"*"	★	star

https://matplotlib.org/3.2.1/api/markers_api.html

Matplotlib base colors:



A tableau palette, CSS colors, and rgb color options are described here:
https://matplotlib.org/stable/gallery/color/named_colors.html

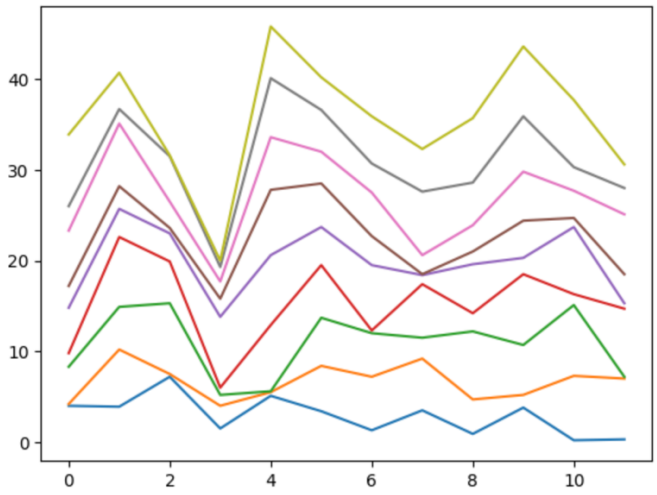
Note: If you're including plots in Python source code files rather than a notebook, you'll want to build your plot then type `plt.show()` to display the graph, then close out of it (top left button) to continue execution.

Example 2a: Recall the ndarry the Woolly Worm activity:

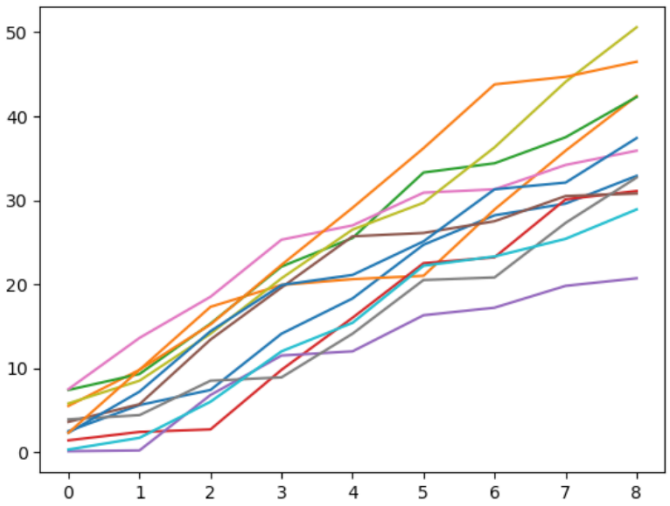
```
totaldist
array([[ 2.5,  5.6,  7.4, 14.1, 18.3, 24.7, 28.2, 29.6, 32.9],
       [ 5.5,  9.8, 17.3, 19.9, 20.6, 21. , 28.9, 35.9, 42.4],
       [ 7.4,  9.3, 15.3, 22.1, 25.5, 33.3, 34.4, 37.5, 42.3],
       [ 1.4,  2.4,  2.7,  9.8, 16. , 22.5, 23.2, 30.1, 31.1],
       [ 0.1,  0.2,  6.8, 11.5, 12. , 16.3, 17.2, 19.8, 20.7],
       [ 3.6,  5.7, 13.4, 19.6, 25.7, 26.1, 27.5, 30.5, 30.8],
       [ 7.5, 13.6, 18.5, 25.3, 27. , 30.9, 31.3, 34.2, 35.9],
       [ 3.9,  4.4,  8.5,  8.9, 14.1, 20.5, 20.8, 27.3, 32.7],
       [ 5.8,  8.5, 14.1, 20.7, 26.5, 29.7, 36.3, 44.1, 50.6],
       [ 0.3,  1.7,  6. , 12. , 15.4, 22.2, 23.3, 25.4, 28.9],
       [ 2.3,  7.2, 14.4, 19.9, 21.1, 25.1, 31.3, 32.1, 37.4],
       [ 2.3,  9.8, 15.2, 22.3, 29.1, 36.2, 43.8, 44.7, 46.5]])
```

Example 2b: What does do each of these tell us?

```
plt.plot(totaldist)
```



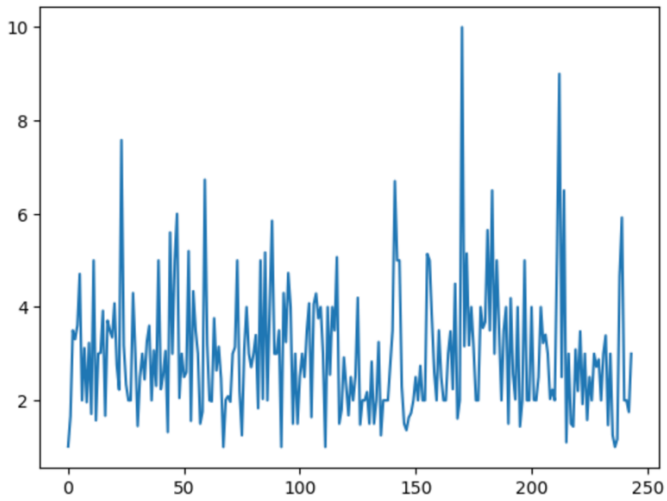

```
plt.plot(totaldist.T)
```



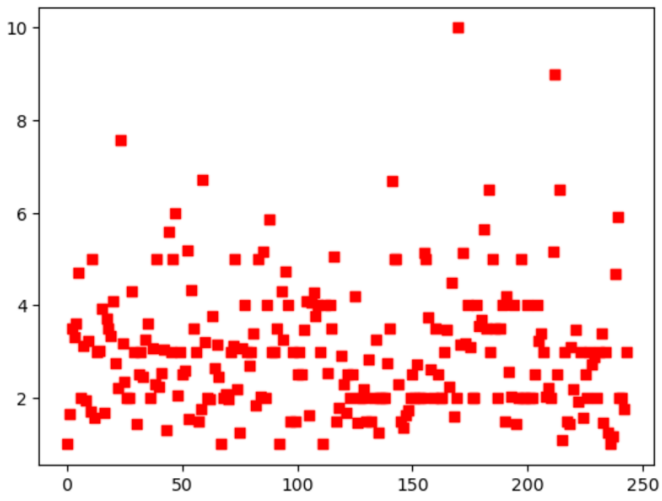
Plotting DataFrames

Example 3a: Since a DataFrame is really a collection of Series, we can plot any Series in a DataFrame. Our `tips` data:

```
plt.plot(tips["tip"])
```




```
plt.plot(tips["tip"], 'rs')
```



What does this tell us?

Example 3b: Since this is not really helpful, let's compute a new column to calculate the tip percentage for each observation:

```
tips['percent'] = tips['tip']/tips['total_bill']*100
```

Then we can use an aggregate function to find the mean of the percentage:

```
tips['percent'].mean()
16.08025817225047
```

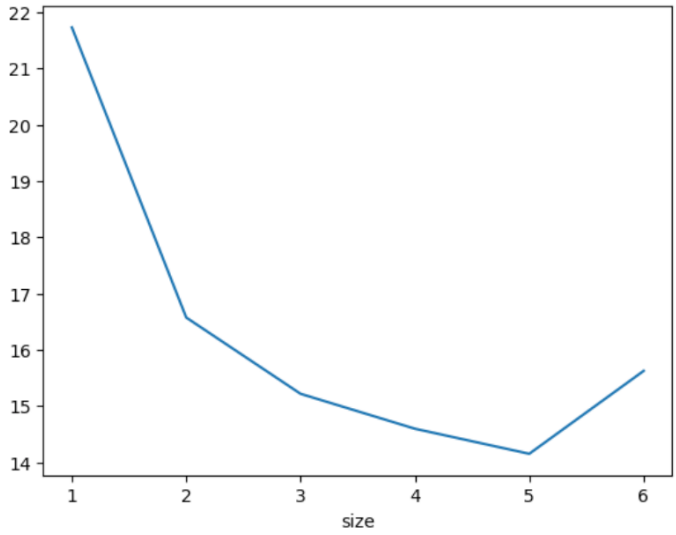
Example 3c: Pandas provides a function to split-apply-combine data by groups according to column(s) or index values. To split the data into groups by party size, then apply the mean function to the percent tip column and put it back together into a Series:

```
groupsize = tips.groupby('size')['percent'].mean()
groupsize
```

```
size
1    21.729202
2    16.571919
3    15.215685
4    14.594901
5    14.149549
6    15.622920
Name: percent, dtype: float64
```

Example 3d: Then plot this Series using pandas or matplotib syntax:

```
groupsize.plot() or plt.plot(groupsize)
```



What does this tell us?

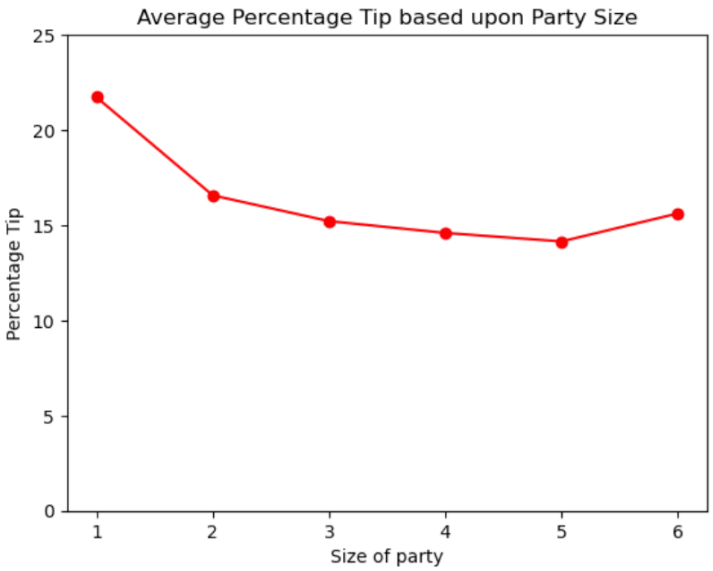
You can group by multiple elements, but must "unstack" to see plots with correct x-axis:

```
myplot = df.groupby([col1, col2, ...])[col3].mean()
myplot.unstack().plot()
```

Customizing Title, Labels, Axis Scale, Legend and Colors

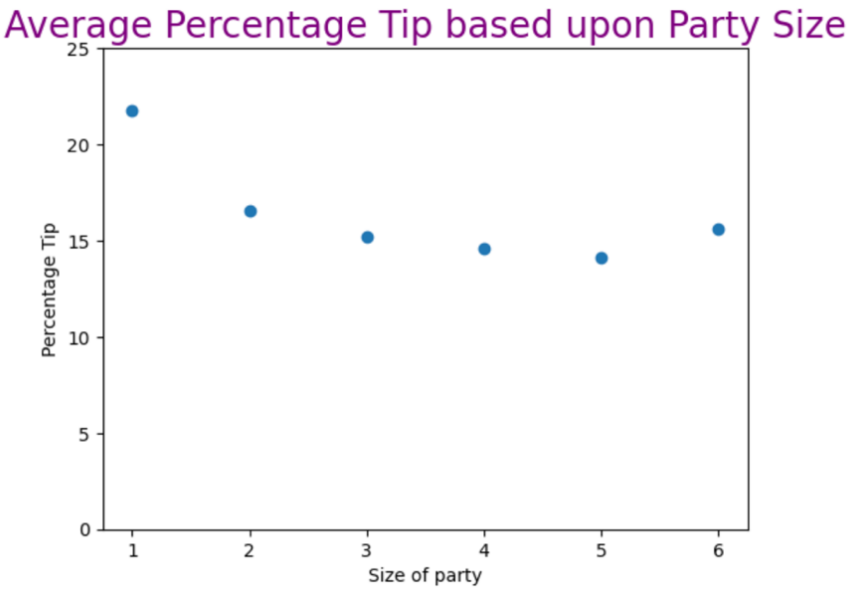
Example 4a: Let's customize our graphs with some additional features. Include all in the same Jupyter notebook code cell:

```
plt.title('Average Percentage Tip based upon Party Size')
plt.xlabel('Size of party')
plt.ylabel('Percentage Tip')
plt.ylim(ymin=0, ymax = 25)
plt.plot(groupsize, color = 'red', marker = 'o')
```



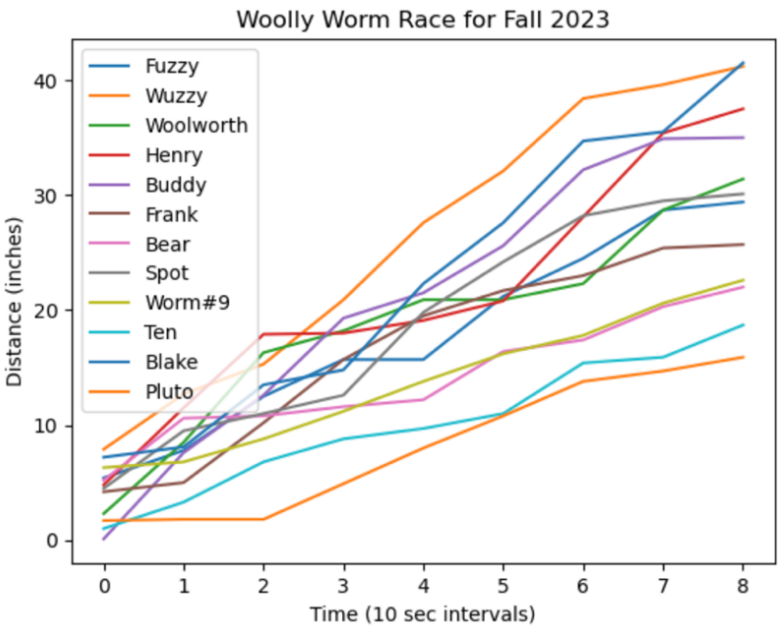
Example 4b: The title method allows for other customizations:

```
plt.title(label = 'Average Percentage Tip based upon Party Size', fontsize = 20, color = 'purple' )
plt.plot(groupsize, 'o')
```



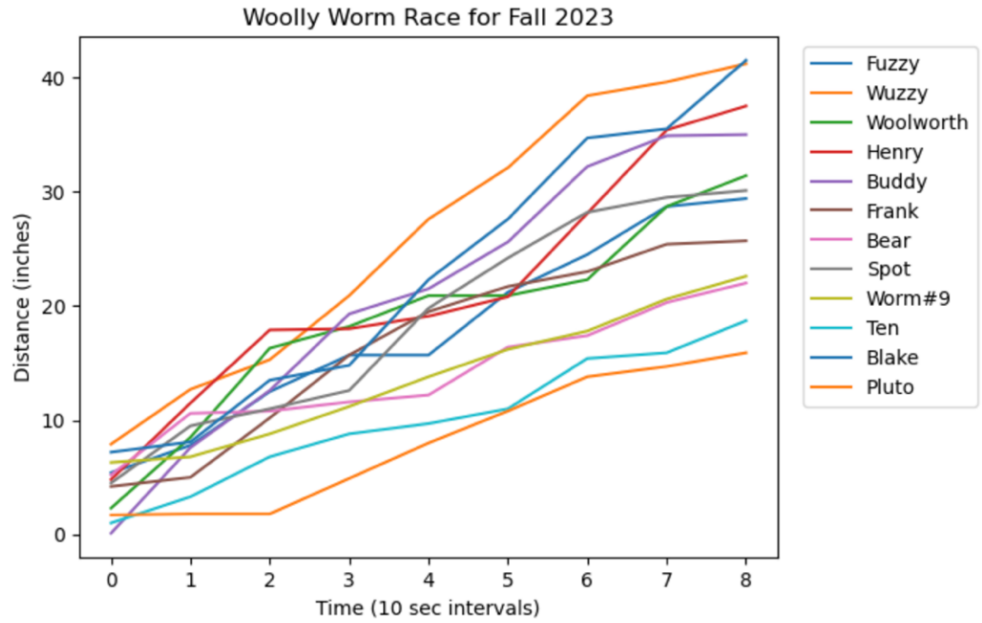
Example 4c: Let's revisit our Woolly Worm plot and add a title, axes labels, and legend:

```
labels = ['Fuzzy', 'Wuzzy', 'Woolworth', 'Henry', 'Buddy', 'Frank', 'Bear', 'Spot','Worm#9',
'Ten', 'Blake', 'Pluto']
plt.legend(labels)
```



Not where you want the legend? Anchor the legend box outside the graph:

```
plt.legend(labels, bbox_to_anchor=(1.3, 1.0), loc = 'upper right')
```



Also try customizing tick marks, tick labels, line styles, markers, etc.

Gapminder – Class Activity

Looking through your initial EDA of the gapminder dataset, what questions might we have:

1)

2)

3)